

EVENT REGISTRATION WEB APPLICATION USING REACT JS

School of Computing

Bachelor of Technology
in
Computer Science & Engineering
By

K Yaswanth Reddy (VTU26049)

10211CS224 - Full Stack Application Development (Java)

Slot: E3-B09



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr. SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “**Event Registration Web Application Using React Js**” submitted by **K Yaswanth Reddy (VTU26049)**, Reg. No. **23UECS0274**, has been carried out in the Winter Semester of Academic Year 2025–2026 under the course **10211CS224 - Full Stack Application Development (Java)**.

Signature of Faculty

Dr.Hafizur Rahman

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

v May, 2026

Signature of Course Coordinator

Dr. M. Sumithra

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in this submission. I understand that any violation of the above will be cause for disciplinary action by the Institute.

K Yaswanth Reddy

VTU26049

Date:06 May, 2026

ABSTRACT

The College Event Registration Portal developed using React JS is a structured web-based application aimed at improving the way departmental events are organized and managed. In many educational institutions, information about events such as hackathons, workshops, seminars, and cultural activities is often shared through informal channels like messaging groups, printed notices, or word of mouth. These approaches can result in poor communication, missed opportunities for students, and inefficient handling of participant data by organizers. To overcome these limitations, the proposed system introduces a unified digital platform that simplifies event discovery, registration, and tracking in a systematic and paperless manner.

The system follows a full-stack development approach, where the frontend is implemented using React JS to provide an interactive and responsive user experience, while the backend is developed using Node.js and Express to handle application logic and API communication. Students can log in using their credentials to explore categorized events, view details such as date, venue, and seat availability, and register seamlessly. Additionally, a calendar interface is integrated to allow users to visualize all the events they have enrolled in. On the administrative side, a dedicated dashboard enables event organizers to add or remove events, define participant limits, and monitor registration statistics in real time. The application uses a MySQL database to store and manage data efficiently, ensuring consistency and reliability even with multiple users accessing the system concurrently [1].

By transitioning from manual methods to an automated event management system, the application minimizes errors such as duplicate entries, overbooking, and data mismanagement. The modular design and component-based architecture of React ensure scalability and ease of maintenance, while RESTful APIs facilitate smooth communication between the frontend and backend layers [3]. Overall, the system enhances user engagement, improves organizational efficiency, and supports a more streamlined and modern approach to handling college events.

Keywords: React JS, Node.js, Express, MySQL, Event Registration, Web Application, Full-Stack Development, REST API, Component-Based Architecture.

LIST OF FIGURES

4.1	System Architecture of Event Registration Portal	17
4.2	Data Flow Diagram of Event Registration Portal	19
4.3	Data Flow Diagram for Ticket Booking System	20
5.1	Event Details and Category-Based Event Display	25
5.2	Event Registration Interface	26
5.3	Calendar View Showing Registered Events	27
5.4	Admin Dashboard with Event Management Controls	27
5.5	Persistent Data Storage in MySQL Database	28

LIST OF TABLES

2.1	Comparison of Existing Event Management Approaches	8
3.1	Database Tables Overview	15
5.1	System Features and Implementation Results	28
7.1	Mapping of Project to Complex Engineering Problem (CEP)	31
7.2	Mapping of Project to Complex Engineering Activities (CEA) . . .	32
7.3	SDG Mapping for the Project	33

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
JSX	JavaScript XML
MERN	MongoDB, Express, React, Node (Full-Stack Architecture)
MySQL	Relational Database Management System
Node.js	JavaScript Runtime Environment
Express	Web Application Framework for Node.js
NPM	Node Package Manager
RDBMS	Relational Database Management System
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UX	User Experience
URL	Uniform Resource Locator

TABLE OF CONTENTS

ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Aim of the Project	3
1.4 Objectives	4
1.5 Scope of the Project	4
1.6 Technologies Used	5
1.7 SDG Alignment	6
2 SURVEY OF SIMILAR APPLICATIONS IN MARKET	7
2.1 Existing Event Management Approaches	7
2.1.1 Notice Boards	7
2.1.2 Messaging Platforms	7
2.1.3 Online Forms	8
2.1.4 Spreadsheet Management	8
2.1.5 External Event Platforms	8
2.2 Comparison of Existing Systems	8
2.3 Need for Proposed System	9

3	FRAMEWORK DESCRIPTION	10
3.1	Frontend Implementation	10
3.1.1	Environment Setup	10
3.1.2	Structure of the Frontend	10
3.1.3	Execution Procedure	11
3.2	Backend Implementation	12
3.2.1	Environment Setup	12
3.2.2	Structure of the Backend	12
3.2.3	Execution Procedure	13
3.3	Database Implementation	13
3.3.1	Database Configuration	13
3.3.2	Schema Structure	14
3.3.3	Database Administration	15
4	METHODOLOGIES	16
4.1	Project Architecture	16
4.1.1	Presentation Layer (Frontend)	16
4.1.2	Application Logic Layer (Backend)	16
4.1.3	Data Access Layer	17
4.1.4	Database Layer	17
4.2	Data Flow Description	17
4.3	Data Flow Diagram	18
4.3.1	Level 0 DFD (Context Diagram)	18
4.3.2	Level 1 DFD	18
4.3.3	Operational Data Flow	19
4.4	Module Descriptions	20
4.4.1	Module 1: Student Authentication Module	20
4.4.2	Module 2: Event Browsing and Search Module	20
4.4.3	Module 3: Event Registration Module	21
4.4.4	Module 4: Calendar Management Module	21
4.4.5	Module 5: Admin Management Module	21
4.4.6	Module 6: Data Management Module	22

4.5	Advantages of the Project	22
4.6	Limitations of the Project	23
5	RESULTS AND DISCUSSIONS	24
5.1	Event Details and Selection	24
5.2	Event Registration Interface	26
5.3	Calendar View of Registered Events	26
5.4	Admin Dashboard Results	27
5.5	Database Implementation Results	28
5.6	Result Summary Table	28
6	CONCLUSION AND FUTURE ENHANCEMENTS	29
6.1	Conclusion	29
6.2	Future Enhancements	30
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	31
7.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	31
7.2	Mapping of the Project to Complex Engineering Activities (CEA)	32
7.3	SDG Mapping (CEP Section)	32
	REFERENCES	34

Chapter 1

INTRODUCTION

1.1 Introduction

The College Event Registration Portal developed using React JS is a dedicated web-based platform aimed at simplifying and organizing the management of departmental events within academic institutions. In most colleges, activities such as hackathons, seminars, workshops, and cultural programs are conducted regularly, yet the coordination of these events often relies on fragmented communication channels such as notice boards and messaging applications, leading to inefficiencies in information delivery [1]. Students may miss important updates or deadlines, while organizers face challenges in maintaining accurate participant records using manual methods like spreadsheets or paper forms [2].

To address these challenges, this project introduces a centralized digital solution that connects students and event organizers through a single, user-friendly interface. The application is built using a modern full-stack approach, where React JS provides a responsive and dynamic frontend, and Node.js with Express handles backend operations and API communication efficiently [3]. Students can log in using their credentials to browse categorized events, view details such as date, venue, and seat availability, and register seamlessly. Additionally, an integrated calendar feature allows users to track all registered events in an organized manner, improving user experience and engagement [4].

All event and registration data are stored in a MySQL database, ensuring reliable data management, consistency, and efficient retrieval even when multiple users access the system simultaneously [5]. The system also supports administrative func-

tionalities, enabling event organizers to create, update, or delete event entries, define participation limits, and monitor registrations in real time [6]. This structured approach helps prevent issues such as overbooking and data redundancy, which are common in manual systems.

By transitioning from traditional methods to an automated event management platform, the application enhances operational efficiency and ensures a transparent, paperless workflow. The use of component-based architecture and RESTful APIs contributes to scalability and maintainability, making the system adaptable for future enhancements and wider institutional use [7]. Ultimately, the platform promotes better coordination, increased student participation, and a more streamlined approach to managing academic events [8].

1.2 Problem Statement

The current management of internal department events relies on manual and semi-digital methods, leading to significant coordination gaps and inefficiencies. The specific problems identified are:

1. **Inconsistent Data Integrity:** In the absence of a structured relational database system, there is no centralized mechanism to associate student information with specific event records. This results in data duplication, inconsistencies, and a high risk of mismatched registration details.
2. **Absence of Self-Service Access:** Students are unable to independently verify their registration status or view previously enrolled events. This dependency on organizers for basic information increases communication overhead and delays.
3. **Limited Search and Filtering Capabilities:** As the number of events increases, students face difficulty in identifying relevant opportunities due to the lack of filtering options based on category, date, or event type.
4. **Ineffective Capacity Control:** Manual registration methods do not support automatic enforcement of participant limits, resulting in overbooking and mismanagement of venue capacity during high-demand events.

5. **Security and Access Limitations:** Common tools such as shared forms do not provide adequate access control, making it difficult to restrict administrative actions and safeguard sensitive student data.
6. **Lack of Analytical Insights:** The absence of an integrated system prevents organizers from generating meaningful reports on participation trends, thereby limiting data-driven decision-making for future events.

Therefore, the development of a centralized and user-friendly Event Registration Portal using React JS is essential to address the existing disconnect between event organizers and students. By adopting a robust full-stack architecture supported by a MySQL database, the system provides a streamlined platform where students can easily explore available events and complete registrations without external dependency. At the same time, administrators are equipped with a secure interface to manage event information, control participant limits, and monitor registrations efficiently. This transition from manual processes to a digital solution ensures improved data accuracy, reduces operational complexity, and supports a more structured and interactive academic event management environment.

1.3 Aim of the Project

The core objective of this project is to design and implement a comprehensive web-based Event Registration System that enhances the efficiency of managing departmental activities. The application leverages React JS to build an interactive and responsive user interface, enabling students to conveniently explore and enroll in various academic and non-academic events. By integrating a structured back-end supported by Node.js and a MySQL database, the system aims to digitize the entire registration workflow, replacing traditional manual methods with a reliable, scalable, and real-time solution. The project emphasizes ease of use, quick access to event information, and seamless coordination between students and organizers, thereby improving overall participation and management efficiency [1].

1.4 Objectives

The key objectives of this project are outlined as follows:

- To develop a fully functional web application using React JS that enables efficient handling of college event registrations through a modern and intuitive interface [2].
- To implement an administrative module that provides authorized users with the ability to create, modify, and remove event details while maintaining data consistency and control [3].
- To utilize MySQL as a relational database system for secure, organized, and persistent storage of event and student-related information [4].
- To design a student-centric interface that allows users to browse events based on categories, view detailed information, and complete registrations with minimal effort [5].
- To incorporate a feature that enables students to track and verify their registered events, improving transparency and reducing dependency on organizers [6].
- To enforce event capacity constraints dynamically, ensuring that registrations are limited according to predefined participant thresholds and preventing overcrowding issues [7].
- To ensure that the application is responsive and accessible across multiple devices, including desktops and mobile platforms, enhancing usability and reach [8].
- To provide basic analytical insights for administrators, enabling them to observe participation trends and evaluate the success of various events for future planning [9].

1.5 Scope of the Project

The proposed Event Registration Portal is intended to support the efficient handling of internal academic events within a departmental environment. The system

primarily focuses on digitizing the process of event discovery, registration, and management through a centralized platform. The scope of the project covers the following aspects:

- **User Interface:** Development of a responsive and interactive web application using **React JS**, enabling students to easily explore categorized events and complete registrations through a seamless interface [1].
- **Administrative Module:** Provision of a dedicated admin panel that allows authorized users to manage event details, control participation limits, and oversee registration data efficiently in real time [2].
- **Data Handling:** Utilization of a **MySQL** database to maintain structured and consistent storage of student information, event records, and registration data, ensuring reliability and data integrity [3].
- **Registration Logic:** Implementation of dynamic seat allocation mechanisms that automatically update availability and restrict registrations once the defined capacity is reached, preventing overbooking scenarios [4].

Although the current implementation focuses on core functionalities such as event management and registration tracking, the system can be further enhanced by incorporating features like automated notifications, attendance verification using QR codes, and detailed user profiles to maintain historical participation data [5].

1.6 Technologies Used

The system is developed using the following technologies:

- **Frontend:** React JS, HTML5, CSS3, JavaScript (ES6+), React Router, Axios for handling API requests and building dynamic user interfaces [6].
- **Backend:** Node.js and Express framework for managing server-side logic and handling RESTful API communication [7].
- **Database:** MySQL, a relational database system used for efficient data storage, retrieval, and management [8].

- **Development Tools:** Visual Studio Code (VS Code), Node.js runtime environment, and npm for package management and project setup [9].
- **Hardware Requirements:** Minimum 4GB RAM, Intel i3 processor or equivalent, and at least 20GB of available storage.
- **Software Requirements:** Windows/Linux/macOS operating system with a modern web browser such as Chrome, Edge, or Firefox.

1.7 SDG Alignment

This project aligns with **Sustainable Development Goal (SDG) 4: Quality Education**, which emphasizes inclusive and equitable access to learning opportunities. By providing a centralized platform for discovering and participating in academic events such as workshops, seminars, and technical competitions, the system enhances student engagement beyond classroom learning [1]. It enables equal access to information for all students, regardless of communication barriers, and promotes skill development through active participation in co-curricular activities [2]. Furthermore, the platform contributes to a more organized academic environment, encouraging continuous learning and knowledge sharing within the institution [3].

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN MARKET

2.1 Existing Event Management Approaches

A preliminary study was conducted to analyze the commonly adopted practices for managing academic events in colleges. The findings indicate that most institutions still rely on traditional or semi-digital approaches, which often lack efficiency, scalability, and proper data handling mechanisms [1]. The widely used methods are discussed below:

2.1.1 Notice Boards

Information regarding events is typically displayed on physical notice boards across the campus. Although this method is easy to implement and does not require technical infrastructure, and there is no mechanism to monitor engagement or participation [2].

2.1.2 Messaging Platforms

Departments often use messaging platforms such as WhatsApp to circulate event details among students. While this enables quick dissemination of information, there is no standardized way to manage registrations or maintain records [3].

2.1.3 Online Forms

Tools like Google Forms are commonly used to collect participant details. These forms provide a basic level of organization by storing responses in spreadsheets. However, they do not offer real-time analytics, automated validations, or a personalized interface for users to track their registrations, resulting in additional manual effort for organizers [4].

2.1.4 Spreadsheet Management

Event coordinators frequently use spreadsheet applications such as Microsoft Excel to maintain participant records. Although useful for storing structured data, these sheets require continuous manual updates and are prone to duplication errors, inconsistencies, and data mismanagement [5].

2.1.5 External Event Platforms

Some institutions adopt third-party event management platforms to handle registrations and ticketing. Tailored for academic use, may include unnecessary functionalities, and can involve subscription costs or service charges [6].

2.2 Comparison of Existing Systems

Table 2.1: Comparison of Existing Event Management Approaches

Method	Advantages	Limitations	Cost
Notice Boards	Easy to implement, no setup required	Limited reach, no tracking mechanism	Free
Messaging Platforms	Fast communication	Unstructured, messages may be missed	Free
Online Forms	Organized data collection	No real-time insights, manual processing	Free
Spreadsheets	Structured storage	Error-prone, requires manual maintenance	Free
Third-Party Platforms	Feature-rich and professional	Costly, not customized for campus needs	Paid
Proposed System	Centralized, automated, real-time updates	Requires initial setup and database configuration	Free

Table 2.2 presents a comparative analysis of existing event management approaches and the proposed system. It highlights how traditional methods lack real-time tracking, structured data handling, and scalability, whereas the proposed system provides a centralized and efficient solution with improved accessibility and automation [2].

2.3 Need for Proposed System

The analysis of existing methods reveals several limitations related to accessibility, data handling, and system efficiency. To overcome these issues, the proposed Event Registration Portal is necessary for the following reasons:

- **Real-Time Updates:** The system ensures that users receive instant information regarding event availability, reducing the risk of overbooking and improving transparency [7].
- **Interactive User Experience:** By leveraging React JS, the application delivers a smooth and responsive interface, enabling users to navigate and register without unnecessary delays or page reloads [8].
- **Centralized Data Storage:** The integration of a MySQL database replaces scattered data sources, ensuring secure, consistent, and easily retrievable records of events and participants [9].
- **Automated Processes:** The system reduces manual intervention by automating registration confirmation and data handling, thereby improving efficiency for both students and administrators [1].
- **Scalability:** Unlike traditional approaches, the web-based system is capable of handling multiple users simultaneously, making it suitable for large-scale events [2].
- **Accessibility:** Being a web application, the system allows users to access event information and register from any location at any time, eliminating dependency on physical presence [3].

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The client-side of the application is implemented using React JS, which supports building reusable UI components and efficient rendering. The development setup includes the following tools:

- **Node.js and NPM:** Provides the runtime environment and dependency management system required to build and run the React application [1].
- **React Router:** Enables client-side navigation, allowing smooth transitions between pages without full reloads [2].
- **Axios:** Used for making asynchronous HTTP requests to interact with backend APIs for fetching and storing data [3].
- **Tailwind CSS:** A utility-first CSS framework used to design a clean and responsive user interface suitable for academic environments [4].
- **VS Code:** Serves as the primary development environment for writing, testing, and debugging frontend components [5].

3.1.2 Structure of the Frontend

The frontend is structured as a Single Page Application (SPA), ensuring faster rendering and better user experience. The organization of files within the `src/`

directory promotes modularity:

- `main.jsx`: Entry point that renders the root component into the DOM.
- `App.jsx`: Acts as the central component managing routing and layout structure of the application.
- `components/`: Contains reusable UI components such as EventCard, Navbar, Sidebar, and Calendar [6].
- `pages/`: Includes major application views like Login, Dashboard, MyEvents, RegisterEvent, and Admin panel.
- `context/`: Stores global state management logic using React Context API for handling user sessions.
- `services/`: Handles API communication logic using Axios for cleaner code separation.
- `hooks/`: Contains custom hooks such as authentication handling to simplify component logic.
- `index.css`: Includes global styling and Tailwind configuration for UI consistency.

3.1.3 Execution Procedure

The frontend follows a component-driven execution model:

1. **Application Load:** When the user accesses the application URL, the browser loads the base HTML file, which triggers the React application initialization [7].
2. **Component Rendering:** The root component initializes routing and displays the appropriate page based on the URL.
3. **User Interaction:** Actions such as selecting events or submitting forms dynamically update the interface without reloading the page.

4. **API Calls:** Axios is used to send requests to the backend server for operations such as login, event fetching, and registration [8].
5. **State Updates:** The Context API manages global state updates, ensuring consistent data flow across all components.

3.2 Backend Implementation

3.2.1 Environment Setup

The server-side of the application is developed using Node.js with the Express framework, enabling efficient handling of client requests and database interactions. The setup includes:

- **Node.js:** Provides the execution environment for server-side JavaScript [9].
- **Express.js:** A lightweight framework used to build RESTful APIs and manage routing.
- **MySQL Driver:** Establishes communication between the server and the relational database.
- **CORS Middleware:** Allows secure cross-origin communication between frontend and backend.
- **Body Parser:** Processes incoming JSON request data for API handling.

3.2.2 Structure of the Backend

The backend is organized in a modular manner to maintain clarity and scalability:

- **server.js:** Main file that initializes the Express server and defines API routes.
- **config/db.js:** Contains database connection logic and query execution functions.
- **routes/:** Includes route definitions for authentication, events, and registrations.

- **controllers/:** Handles business logic for processing requests and sending responses.
- **models/:** Represents database schema interactions for structured data handling.

3.2.3 Execution Procedure

The backend operates through the following steps:

1. **Server Initialization:** The application is started using commands such as `node server.js` or `npm run dev`.
2. **Configuration Loading:** Environment variables and configurations are loaded to set up database connections and server ports.
3. **Database Connection:** The server establishes a connection with the MySQL database to handle persistent data operations.
4. **Request Handling:** Incoming API requests are routed to appropriate controllers based on defined endpoints.
5. **Response Delivery:** The server processes the request, interacts with the database if required, and sends responses back to the frontend application.

3.3 Database Implementation

3.3.1 Database Configuration

The application uses **MySQL** as the primary database system to ensure reliable and structured data storage for events and registrations. The connection between the Node.js backend and the database is established using the `mysql2` package within the `config/db.js` file. The configuration includes the following parameters:

- **Host:** Localhost or the designated database server address.
- **Database Name:** `event_portal_db`
- **User Credentials:** Stored securely using environment variables in a `.env` file.

- **Connection Pooling:** Enabled to efficiently manage multiple concurrent database requests from users [1].

3.3.2 Schema Structure

The database schema is designed to support event management and student registrations efficiently. It consists of three primary tables that maintain relationships between users and events, as summarized in Table 3.1.

STUDENTS Table

This table stores essential information related to students accessing the system.

- `id`: Unique identifier (Primary Key).
- `reg_no`: Registration number used for login authentication.
- `dob`: Date of birth used as a secondary authentication factor.
- `name / dept / year / section`: Basic academic details of the student.

EVENTS Table

This table maintains records of all events organized within the department.

- `id`: Primary key for event identification.
- `title`: Name of the event.
- `category`: Classification such as hackathon, cultural, or sports.
- `date / venue`: Schedule and location details.
- `capacity`: Maximum number of participants allowed.

REGISTRATIONS Table

This Table:3.1 shows the mapping between students and events, storing registration data.

- `id`: Primary key.
- `student_id`: Foreign key referencing the STUDENTS table.
- `event_id`: Foreign key referencing the EVENTS table.

Table 3.1: Database Tables Overview

Table Name	Purpose
<code>students</code>	Stores login credentials and academic details of users.
<code>events</code>	Maintains information about all available events including schedule and capacity.
<code>registrations</code>	Records student participation by linking users with events.

3.3.3 Database Administration

The MySQL database is managed using tools such as MySQL Workbench, which provides a graphical interface for executing queries, managing tables, and monitoring data. This ensures that all event and registration records are persistently stored and remain unaffected even if the backend server is restarted. The use of a relational database system also supports efficient querying, indexing, and data integrity for large-scale usage [2].

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The overall design of the Event Registration Portal follows a modular and layered architecture, enabling clear separation between the frontend and backend systems. This structure ensures scalability, maintainability, and efficient communication through RESTful APIs. The architectural representation of the system is illustrated in Fig: 4.1, which highlights the interaction between different layers of the application [1].

4.1.1 Presentation Layer (Frontend)

The presentation layer is implemented using **React JS**, providing an interactive and responsive user interface. It consists of reusable components such as event cards, navigation bars, and calendar views that allow users to browse events, perform registrations, and manage their activities. User actions trigger asynchronous API calls using **Axios**, ensuring smooth and dynamic updates without full page reloads [2].

4.1.2 Application Logic Layer (Backend)

The backend layer is developed using **Node.js** and **Express.js**, which handle the core processing logic of the system.

- **Routing:** API endpoints defined in `server.js` manage incoming requests and direct them to appropriate handlers.

- **Business Logic:** Validates user inputs, enforces event capacity constraints, and processes registration requests.
- **Control Flow:** Ensures proper coordination between frontend requests and database operations [3].

4.1.3 Data Access Layer

This layer acts as an interface between the application logic and the database. It utilizes the **mysql2** library to execute structured SQL queries for performing CRUD operations. The layer ensures efficient data retrieval and insertion while maintaining consistency across the system [4].

4.1.4 Database Layer

The database layer is powered by **MySQL**, which stores all persistent data including student records, event details, and registration information. The relational structure enables efficient querying and ensures referential integrity between different entities [5].



Figure 4.1: System Architecture of Event Registration Portal

Fig: 4.1 illustrates the layered architecture of the system, showing how the frontend communicates with the backend through APIs, and how the backend interacts with the database to process and store information.

4.2 Data Flow Description

The flow of data within the system follows a structured sequence to ensure accurate processing and response generation:

1. **User Interaction:** The student logs in and selects an event through the React-based interface.
2. **Request Handling:** The frontend sends a request to the backend server via API endpoints.
3. **Processing:** The backend validates the request, checks event availability, and processes registration logic.
4. **Database Operation:** Relevant data is stored or retrieved from the MySQL database.
5. **Response Delivery:** The server sends a response back to the frontend, updating the user interface accordingly [6].

4.3 Data Flow Diagram

The Data Flow Diagram (DFD) represents how information moves through the system, highlighting interactions between users, processes, and data stores. Fig: 4.2 provides a visual representation of these flows [7].

4.3.1 Level 0 DFD (Context Diagram)

At the highest level, the system is treated as a single process interacting with external entities:

- **Student:** Accesses the system to view events and perform registrations.
- **Admin:** Manages event data and monitors registration activities.

4.3.2 Level 1 DFD

The Level 1 DFD expands the system into detailed processes:

- **Process 1: Event Viewing:** Retrieves event details from the database and displays them to users.
- **Process 2: Registration Handling:** Validates and records student registrations.

- **Process 3: Capacity Management:** Ensures that registrations do not exceed predefined limits.
- **Process 4: Data Retrieval:** Provides registered event details for user tracking and display.
- **Data Store 1:** `students` table
- **Data Store 2:** `events` table
- **Data Store 3:** `registrations` table



Figure 4.2: Data Flow Diagram of Event Registration Portal

Fig: 4.2 depicts the logical flow of data within the system, showing how user inputs are processed, validated, and stored, and how results are delivered back to the user interface.

4.3.3 Operational Data Flow

The step-by-step operational flow of the system is described below:

1. **Input Stage:** The student logs in and selects an event from the dashboard interface.
2. **Processing Stage:** The backend validates the request and checks availability constraints.
3. **Storage Stage:** Registration details are recorded in the database.
4. **Output Stage:** The system updates the user interface and reflects the registered events in the calendar view.

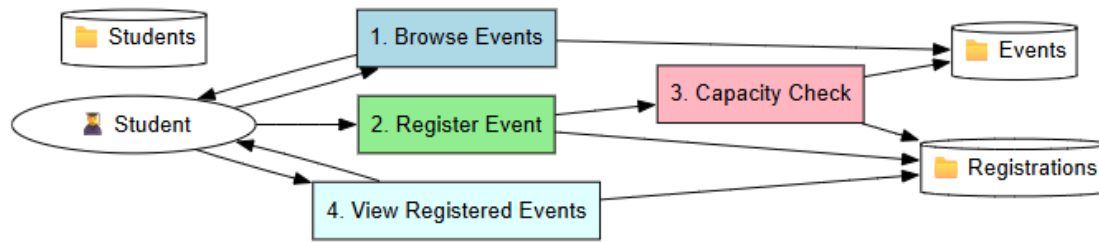


Figure 4.3: Data Flow Diagram for Ticket Booking System

The above Fig: 4.3 illustrates the Data Flow Diagram of the Event Registration Portal, showing how data moves between students, administrators, the application server, and the database. It highlights key processes such as event browsing, registration handling, and data storage, ensuring a clear understanding of system interactions and workflow [3].

4.4 Module Descriptions

4.4.1 Module 1: Student Authentication Module

This module handles the login functionality for students using their academic credentials instead of traditional OTP-based systems.

- **Functionality:** Students log in using their registration number and date of birth, which are verified against the database records.
- **Validation:** Ensures only authorized students can access the platform and perform event registrations.
- **Session Handling:** Maintains login state using React Context API for seamless navigation across pages [1].

4.4.2 Module 2: Event Browsing and Search Module

This module provides students with the ability to explore and filter available events.

- **Dynamic UI:** Displays events as interactive cards categorized into technical, cultural, sports, and other domains.
- **Search Functionality:** Allows users to quickly find events using keywords or categories.
- **Carousel Display:** Events are presented in a sliding card format to enhance visual experience and usability [2].

4.4.3 Module 3: Event Registration Module

This module is responsible for handling event enrollment by students.

- **Registration Logic:** Enables students to register for multiple events while ensuring participant limits are not exceeded.
- **Validation:** Checks for duplicate registrations and enforces event capacity constraints.
- **Database Integration:** Stores registration data in the `registrations` table for persistent tracking [3].

4.4.4 Module 4: Calendar Management Module

This module provides a visual representation of registered events.

- **Calendar View:** Displays all events a student has registered for in a structured calendar format.
- **Event Removal:** Allows users to cancel registrations directly from the calendar interface.
- **User Interaction:** Clicking on a specific date reveals event details and actions [4].

4.4.5 Module 5: Admin Management Module

This module enables administrators to control and manage the entire system.

- **Event Control:** Admins can create, update, and delete events.
- **Monitoring:** Provides real-time count of registrations for each event.
- **Capacity Handling:** Allows setting default participant limits (e.g., 250 per event) [5].

4.4.6 Module 6: Data Management Module

This module ensures efficient storage and retrieval of application data.

- **Database Operations:** Handles CRUD operations using MySQL queries.
- **Data Integrity:** Maintains consistency between students, events, and registrations.
- **Performance:** Optimized queries ensure fast response times even with multiple users [6].

4.5 Advantages of the Project

- **Unified Platform:** Provides a single system for managing all departmental events, eliminating scattered communication channels [7].
- **Reliable Data Storage:** Utilizes MySQL to ensure secure and long-term storage of event and registration data [8].
- **Controlled Access:** Authentication mechanism ensures only valid users can access the system and perform operations.
- **Real-Time Updates:** Automatically updates event availability, preventing overbooking and ensuring accuracy [9].
- **Improved User Experience:** React-based interface enables smooth navigation without page reloads, enhancing usability.
- **Interactive Features:** Includes calendar visualization and sliding event cards for better engagement.

- **Administrative Efficiency:** Reduces manual workload by automating event management and participant tracking.
- **Accessibility:** Being a web-based system, it can be accessed from any device with internet connectivity.

4.6 Limitations of the Project

- **Authentication Simplicity:** The current login mechanism is based on basic credential validation (registration number and date of birth), which may not provide advanced security features such as multi-factor authentication [1].
- **Limited Intelligent Assistance:** The system does not include an advanced AI-based assistant, and hence cannot handle complex user queries or provide conversational support dynamically [2].
- **Absence of Payment Integration:** The application currently supports only free event registrations and does not include any integrated payment gateway for handling paid events [3].
- **Manual Attendance Handling:** There is no automated attendance tracking mechanism such as QR-code scanning, requiring manual verification during events [4].
- **Restricted Admin Roles:** The system supports a single administrator role and does not yet provide multi-level access control or role-based permissions [5].
- **No Data Export Facility:** Administrators cannot directly export registration data into formats such as PDF or Excel for reporting purposes [6].
- **Platform Dependency:** The application is limited to web browsers and does not have a dedicated mobile application for Android or iOS platforms [7].

Chapter 5

RESULTS AND DISCUSSIONS

5.1 Event Details and Selection

The system provides a dedicated interface where users can explore detailed information about each event, including description, category, date, and venue. The interface dynamically updates based on user interaction, ensuring a smooth browsing experience without page reloads. Fig: 5.1 illustrates the event exploration and selection interface, where multiple categorized events are displayed using card-based layouts [1].

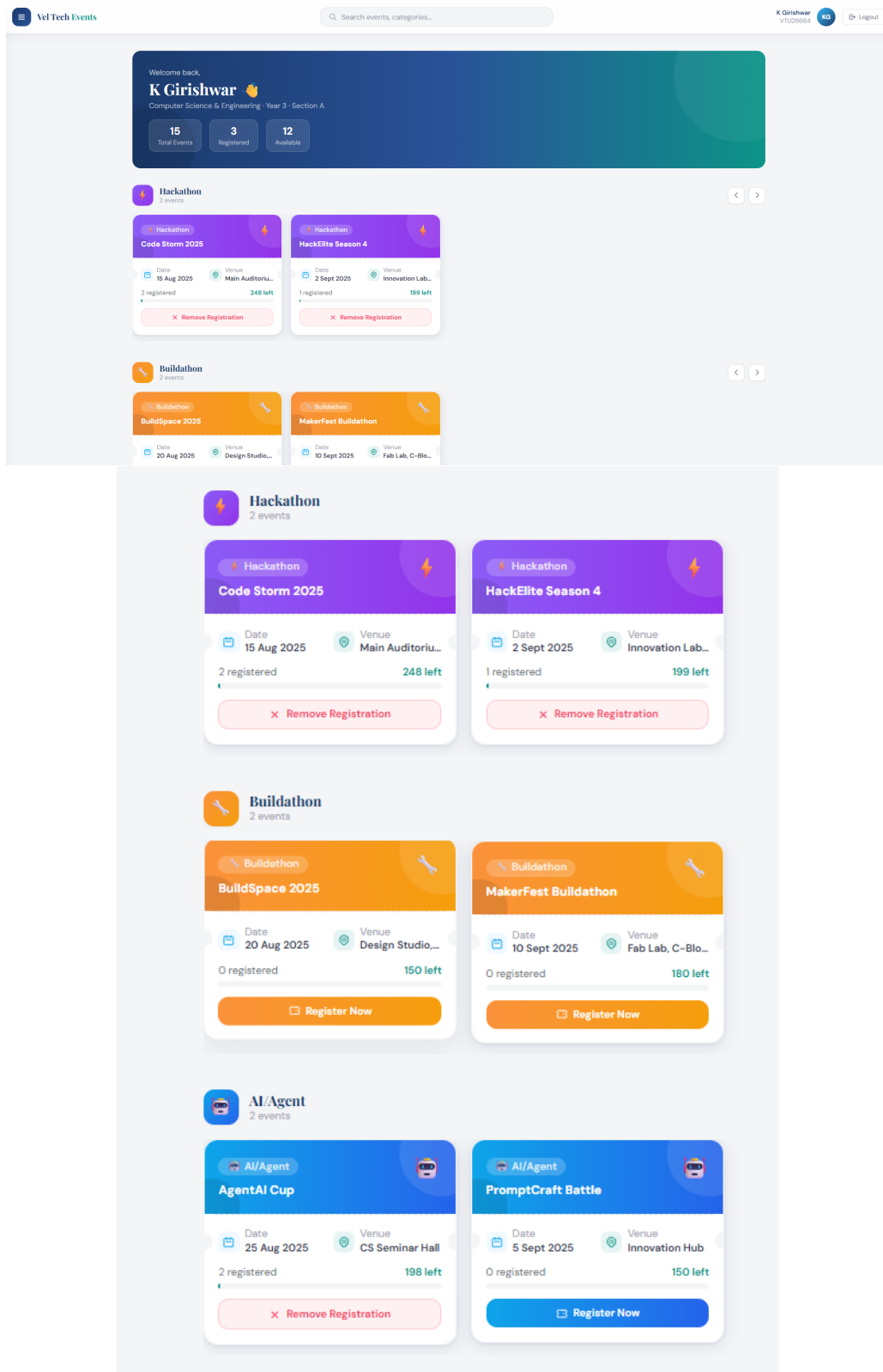
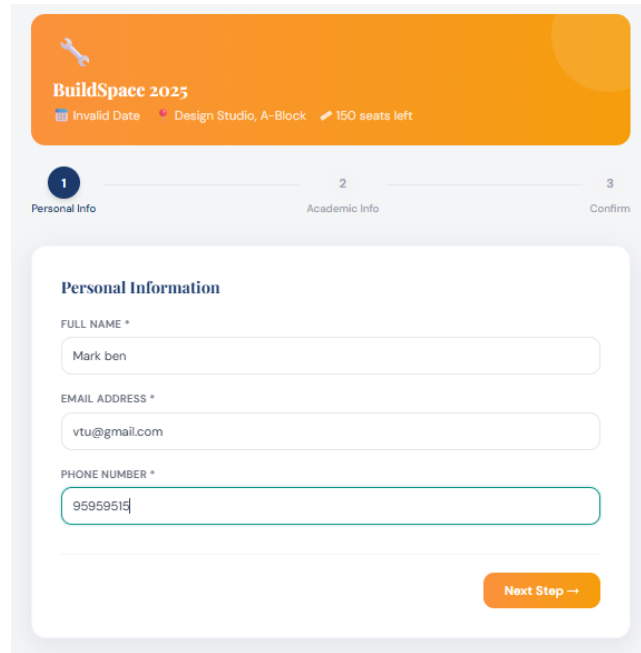


Figure 5.1: Event Details and Category-Based Event Display

5.2 Event Registration Interface

When a student selects an event, the system provides a simple registration interface where the user can confirm participation. The system validates inputs and ensures that the event capacity is not exceeded. Fig: 5.2 shows the event registration interface with user interaction flow [2].



The image displays a web-based event registration interface for 'BuildSpace 2025'. At the top, an orange banner features the event title and a progress bar with three steps: '1 Personal Info', '2 Academic Info', and '3 Confirm'. Below the banner, the 'Personal Information' section contains three input fields: 'FULL NAME *' with the value 'Mark ben', 'EMAIL ADDRESS *' with 'vtu@gmail.com', and 'PHONE NUMBER *' with '95959515'. A green border highlights the phone number field. An orange 'Next Step →' button is positioned at the bottom right of the form.

Figure 5.2: Event Registration Interface

5.3 Calendar View of Registered Events

After successful registration, the system updates the user's personal calendar with the selected events. This feature allows users to track all their registered activities in an organized manner. Fig: 5.3 demonstrates how registered events are visually represented within the calendar module [3].

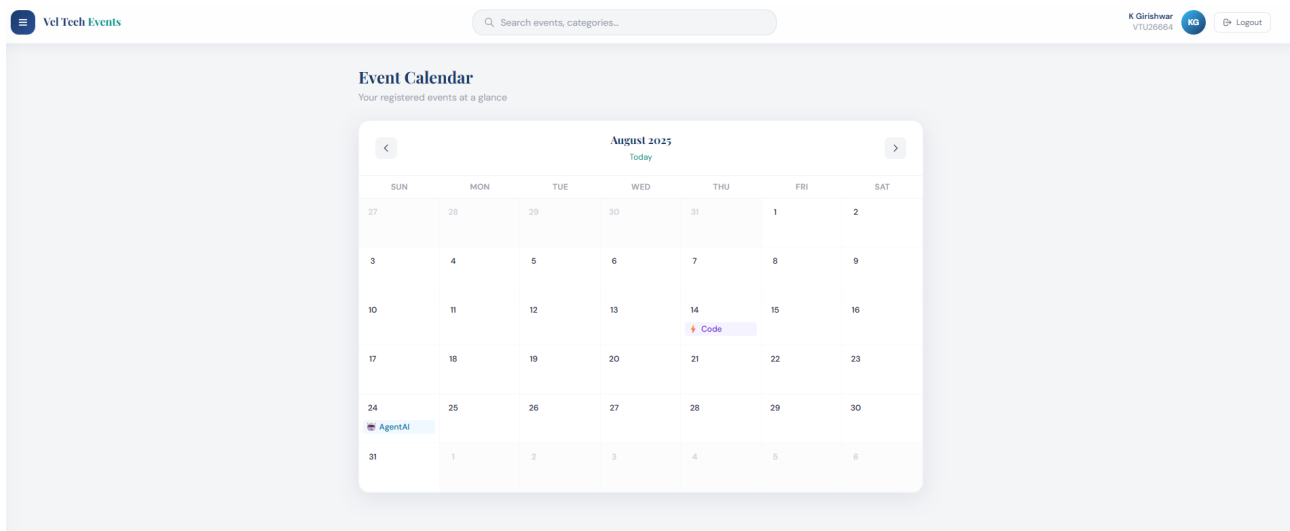


Figure 5.3: Calendar View Showing Registered Events

5.4 Admin Dashboard Results

The administrative interface enables event organizers to manage event data efficiently. It provides features such as adding new events, deleting existing ones, and monitoring participant counts. Fig: 5.4 presents the admin dashboard with real-time event statistics and controls [4].

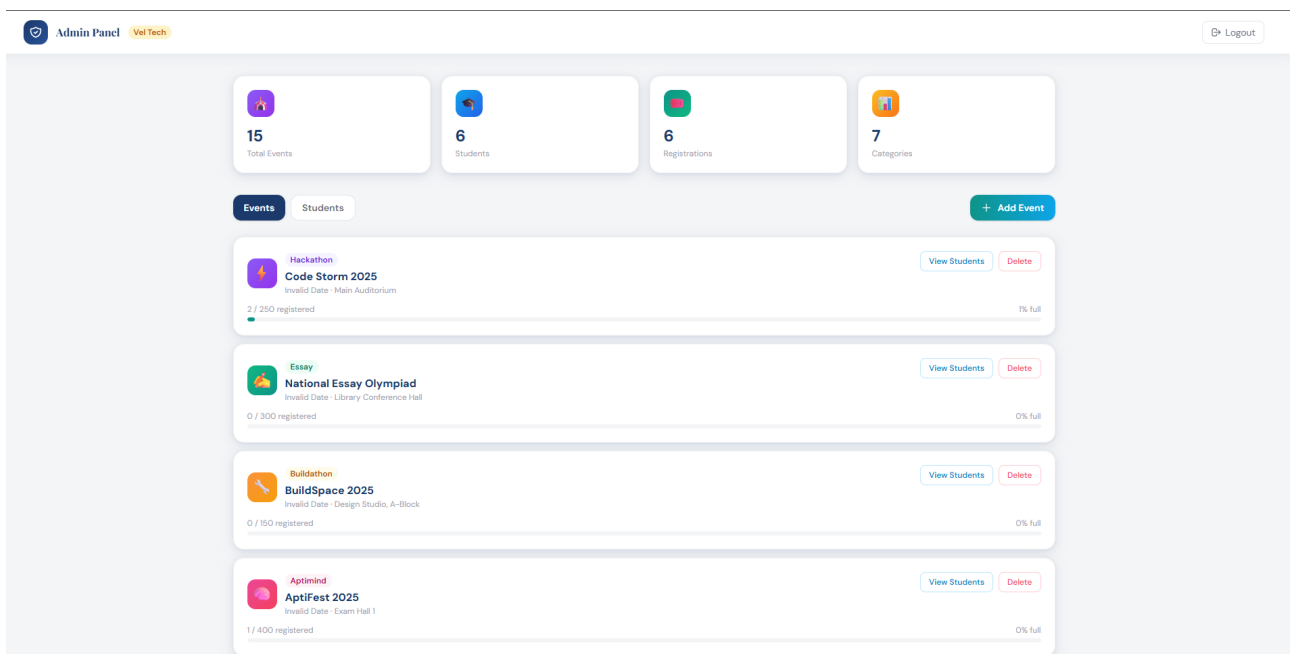
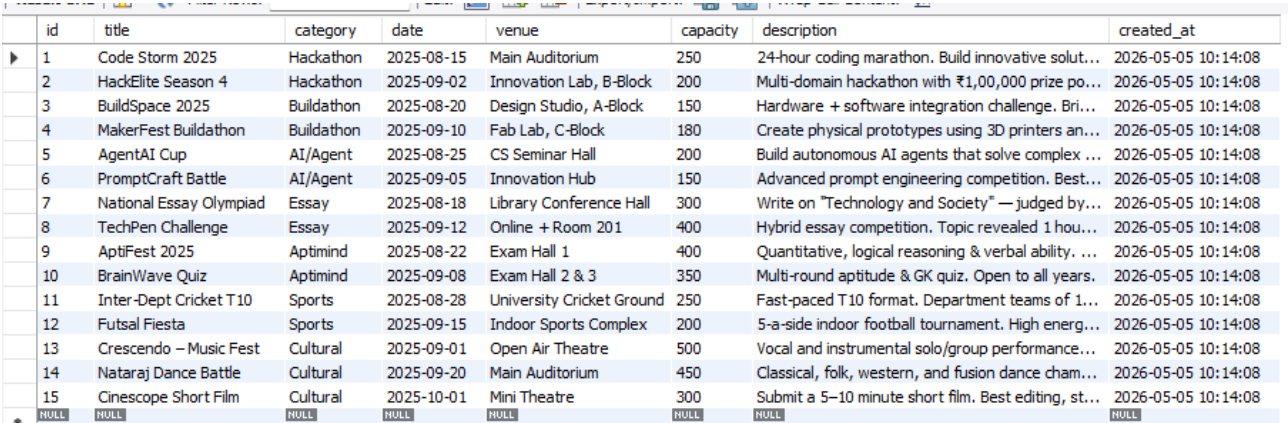


Figure 5.4: Admin Dashboard with Event Management Controls

5.5 Database Implementation Results

The backend successfully stores all relevant data in the MySQL database, ensuring persistence and reliability. Event details and student registrations are maintained in structured tables, allowing efficient querying and retrieval. Fig: 5.5 shows the database records as viewed in MySQL Workbench [5].

A screenshot of the MySQL Workbench interface showing a table with 15 rows of event data. The table has columns for id, title, category, date, venue, capacity, description, and created_at. The data includes various events like Code Storm 2025, HackElite Season 4, BuildSpace 2025, MakerFest Buildathon, AgentAI Cup, PromptCraft Battle, National Essay Olympiad, TechPen Challenge, AptiFest 2025, BrainWave Quiz, Inter-Dept Cricket T10, Futsal Fiesta, Crescendo – Music Fest, Nataraj Dance Battle, and Cinescope Short Film. The last row shows NULL values for all columns.

id	title	category	date	venue	capacity	description	created_at
1	Code Storm 2025	Hackathon	2025-08-15	Main Auditorium	250	24-hour coding marathon. Build innovative solut...	2026-05-05 10:14:08
2	HackElite Season 4	Hackathon	2025-09-02	Innovation Lab, B-Block	200	Multi-domain hackathon with ₹1,00,000 prize po...	2026-05-05 10:14:08
3	BuildSpace 2025	Buildathon	2025-08-20	Design Studio, A-Block	150	Hardware + software integration challenge. Bri...	2026-05-05 10:14:08
4	MakerFest Buildathon	Buildathon	2025-09-10	Fab Lab, C-Block	180	Create physical prototypes using 3D printers an...	2026-05-05 10:14:08
5	AgentAI Cup	AI/Agent	2025-08-25	CS Seminar Hall	200	Build autonomous AI agents that solve complex ...	2026-05-05 10:14:08
6	PromptCraft Battle	AI/Agent	2025-09-05	Innovation Hub	150	Advanced prompt engineering competition. Best...	2026-05-05 10:14:08
7	National Essay Olympiad	Essay	2025-08-18	Library Conference Hall	300	Write on "Technology and Society" — judged by...	2026-05-05 10:14:08
8	TechPen Challenge	Essay	2025-09-12	Online + Room 201	400	Hybrid essay competition. Topic revealed 1 hou...	2026-05-05 10:14:08
9	AptiFest 2025	Aptimind	2025-08-22	Exam Hall 1	400	Quantitative, logical reasoning & verbal ability. ...	2026-05-05 10:14:08
10	BrainWave Quiz	Aptimind	2025-09-08	Exam Hall 2 & 3	350	Multi-round aptitude & GK quiz. Open to all years.	2026-05-05 10:14:08
11	Inter-Dept Cricket T10	Sports	2025-08-28	University Cricket Ground	250	Fast-paced T10 format. Department teams of 1...	2026-05-05 10:14:08
12	Futsal Fiesta	Sports	2025-09-15	Indoor Sports Complex	200	5-a-side indoor football tournament. High energ...	2026-05-05 10:14:08
13	Crescendo – Music Fest	Cultural	2025-09-01	Open Air Theatre	500	Vocal and instrumental solo/group performance...	2026-05-05 10:14:08
14	Nataraj Dance Battle	Cultural	2025-09-20	Main Auditorium	450	Classical, folk, western, and fusion dance cham...	2026-05-05 10:14:08
15	Cinescope Short Film	Cultural	2025-10-01	Mini Theatre	300	Submit a 5–10 minute short film. Best editing, st...	2026-05-05 10:14:08
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 5.5: Persistent Data Storage in MySQL Database

5.6 Result Summary Table

The overall implementation outcomes and system performance are summarized in Table 5.1.

Table 5.1: System Features and Implementation Results

Feature	Result
Frontend Framework	React JS enables a fast and responsive Single Page Applica-tion experience.
User Authentication	Credential-based login ensures controlled system access.
Database Management	MySQL ensures secure and persistent storage of application data.
Event Registration	Real-time updates prevent overbooking and ensure accurate participation tracking.
Calendar Integration	Registered events are visually organized for better user expe-rience.
Admin Control	Dashboard provides complete control over event lifecycle management.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Event Registration Portal developed using React JS has been successfully implemented as a full-stack web application that simplifies and digitizes the management of departmental events. The system integrates a responsive frontend with a Node.js and Express.js backend, delivering a smooth and efficient Single Page Application experience for users [1].

The application effectively meets its intended goals by providing a centralized platform where students can explore categorized events, register seamlessly, and manage their participation through features such as calendar tracking. The use of a structured MySQL database ensures reliable storage and retrieval of event and registration data, thereby eliminating inconsistencies associated with manual record-keeping methods [2].

Additionally, the system introduces administrative capabilities that allow event organizers to control event listings, monitor participation counts, and enforce capacity limits. The overall architecture demonstrates the practicality of combining modern frontend technologies with efficient backend services to build scalable academic solutions. The project also contributes to enhancing student engagement and aligns with digital transformation initiatives in educational environments, supporting the objectives of SDG 4 (Quality Education) [3].

6.2 Future Enhancements

Although the current system fulfills its core requirements, several improvements can be incorporated to extend its functionality and usability:

- **Payment Integration:** Introduce secure payment gateways such as Razorpay or Stripe to support paid event registrations and automate transaction handling [4].
- **QR-Based Attendance System:** Implement QR code generation for each registration to enable quick and automated attendance verification during events [5].
- **Enhanced Notification System:** Add email and SMS notification services to inform users about registration confirmations, reminders, and updates [6].
- **Advanced Analytics Dashboard:** Develop graphical dashboards for administrators to analyze participation trends and event performance metrics [7].
- **Role-Based Access Control:** Introduce multiple admin roles with different permission levels to improve system security and management flexibility [8].
- **Export Functionality:** Provide options to download registration data in formats such as PDF or Excel for reporting and documentation purposes [9].
- **Mobile Application Development:** Extend the system into a mobile application using frameworks like React Native to improve accessibility and user convenience.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Knowledge Depth	Application of modern technologies	WK4, WK5	Involves the use of React JS, back-end APIs, and database integration for building a scalable system.
WP2	Design Constraints	Balancing usability and control	WK4, WK5	Ensures a user-friendly interface while maintaining secure and structured data handling.
WP3	Analytical Thinking	System design and logic flow	WK3, WK4	Requires planning of component structure, state management, and efficient data flow.
WP4	Problem Nature	Semi-structured real-world problem	WK4	Addresses inefficiencies in traditional event management methods through automation.
WP5	Solution Approach	Customized implementation	WK6	Develops tailored logic for event registration, validation, and capacity management.
WP6	Stakeholder Involvement	Multiple user roles	WK5, WK6	Caters to both students and administrators with different functionalities.
WP7	System Integration	Multi-layer architecture	WK3, WK5	Combines frontend, backend, and database layers into a unified working system.

Table 7.1 presents the mapping of the developed Event Registration Portal to the characteristics of Complex Engineering Problems (CEP). It highlights how the project involves modern technologies, multi-layer system integration, analytical design thinking, and the ability to address real-world challenges in event management. The system effectively balances usability, scalability, and functionality while meeting the requirements of multiple stakeholders such as students and administrators [4].

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes React JS, HTML, CSS, and modern web frameworks for development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event display, booking, and validation modules.
EA3	Innovation	Application of modern technologies	Implements a web-based platform with React Hooks and conditional rendering.
EA4	Consequences to Society	Impact on users and society	Enhances student engagement and simplifies the booking process while reducing delays.

Table 7.2 illustrates the mapping of the developed system to Complex Engineering Activities (CEA). It demonstrates how the project incorporates modern technologies, handles multiple interacting modules, and delivers innovative solutions that improve user experience while contributing positively to academic environments [4].

7.3 SDG Mapping (CEP Section)

This Table:7.3 highlights how the developed Event Registration Portal aligns with global sustainability objectives while addressing complex engineering challenges and user-centric design [4]. The mapping presented above reflects the project's

Table 7.3: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Enhances accessibility to academic events such as workshops, seminars, and hackathons, enabling students to improve skills and knowledge through a centralized digital platform [1].
SDG 9	Industry, Innovation and Infrastructure	Encourages digital transformation in educational institutions by implementing a scalable React-based web system integrated with backend services and databases [2].
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by reducing paper usage, digitizing event workflows, and improving resource planning and participation tracking [3].

contribution toward improving educational accessibility, promoting digital infrastructure, and supporting sustainable institutional practices [5].

REFERENCES

1. **React Official Documentation.** "Getting Started with React for Web Applications." <https://react.dev/learn>
2. **Node.js Documentation.** "Node.js v20.x Manual and Documentation for Back-end Development." <https://nodejs.org/docs/latest/api/>
3. **MySQL Reference Manual.** "Relational Database Management and SQL Syntax." <https://dev.mysql.com/doc/refman/8.0/en/>
4. **Express.js Framework.** "Fast, unopinionated, minimalist web framework for Node.js." <https://expressjs.com/>
5. **Tailwind CSS Documentation.** "Utility-First CSS Framework for Rapid UI Development." <https://tailwindcss.com/docs>
6. **ResearchGate (2026).** "Web-Based Event Registration System using React and MySQL." <https://www.researchgate.net>
7. **IJEDR (2026).** "Event Management System: A Full-Stack Development Approach." <https://ijedr.org>
8. **IEEE Xplore (2025).** "Performance Analysis of React-based Single Page Applications." <https://ieeexplore.ieee.org>
9. **United Nations.** "Sustainable Development Goal 4: Quality Education." <https://sdgs.un.org/goals/goal4>